Azure Functions+TypeScript

2018/4/14第二十一回 Azureもくもく会 @ 品川

大中浩行(@setoazusa)

Copyright 2017 Hiroyuki Onaka

こんにちわ！

- 大中浩行 / setoazusa(せとあず)
- 開発とインフラの二刀流
- Javaの方から来ました
- 4/22 技術書典4
  か-16「ふぃーるどの一つ」
  ＆ くー35「錬金術MeetUp」

Copyright 2017 Hiroyuki Onaka

今日やったこと

Azure Fucntions上でTypeScriptで書いたバッチ処理を動かす

Copyright 2017 Hiroyuki Onaka

やること

- JavaScriptの Functionをつくる
- TypeScriptでスクリプトを書く
- GitレポジトリーとFunctionsを連携する
- デプロイ時にtscコマンドによるコンパイルが走るようにする

Copyright 2017 Hiroyuki Onaka

JavaScriptのFunctionをつくる

Azure CLI で初めての関数を作成する

(https://docs.microsoft.com/ja-jp/azure/azure-functions/functions-create-first-azure-function-azure-cli)にしたがって関数を作成した後、index.jsを削除してindex.tsを作成する。

Copyright 2017 Hiroyuki Onaka

TypeScriptでスクリプトを書く(tsconfig.jsonの抜粋)

```
1   {
2     "compilerOptions": {
3       "target": "ES2015",
4       "module": "commonjs",
5       "lib": [ "es2015" ],
6       "allowJs": true,
7       "sourceMap": true,
8       "rootDir": "./",
9     },
10    "exclude": [
11      "node_modules"
12    ]
13  }
```

CC BY

GitレポジトリーとFunctionを連携

Azure Portal上でGithub/Bitbucketと連携を設定することで、Gitのリモートレポジトリーにpushするとと Azure Functionsにコードがデプロイされます。

Copyright 2017 Hiroyuki Onaka

デプロイ時にtscコマンドによるコンパイルが走るようにする

- Azureにデプロイ時にnpm installが実行される
- npm installの処理時にtscコマンドによるコンパイルが実行されるようにする

Copyright 2017 Hiroyuki Onaka

package.jsonのpostinstallの処理で記述

```
16      },
17      "dependencies": {
18        "cheerio-httpcli": "^0.7.2",
19        "request-promise": "^4.2.2",
20        "tough-cookie": "^2.3.4"
21      },
22      "scripts": {
23        "postinstall": "npm install -g typescript && tsc"
24      }
25    }
26
```

M

Azure FunctionsのtimeTriggerの注意点

エクスプローラー

開いているエディター
- tsconfig.json M
- function.json
- package.json
- index.ts

TIMERTRIGGERJS
- bin
- functions-extensions
- node_modules
- function.json
- index.js
- index.js.map
- index.ts
- package-lock.json
- package.json
- readme.md
- sample.dat
- tsconfig.json M

tsconfig.json | function.json | package.json | index.ts

```
{
  "disabled": false,
  "bindings": [
    {
      "name": "myTimer",
      "type": "timerTrigger",
      "direction": "in",
      "schedule": "0 10 * * * *"
    }
  ]
}
```

ここは「秒」

作ったもの

- 技術書典のサークル被チェック数を取得して、サーバー監視サービスのMackerelにメトリックを投稿するもの
- https://github.com/azusa/circlecheck-mackerel
- ベースはerukitiさんの https://github.com/erukiti/techbookfest-circle-bot

Copyright 2017 Hiroyuki Onaka

コード(1)

```
6   const email = process.env["EMAIL"]
7   const password = process.env["PASSWORD"]
8   const webhook = process.env.SLACK_WEBHOOK
9   const channel = process.env.SLACK_CHANNEL
10  const apiKey = process.env["MACKREL_API_KEY"]
11
12  export async function run (context: any, mytimer: any) {
13    const checkedCount = await sendCheckedcountToSlack()
14    console.log(new Date().toLocaleString(), checkedCount)
15    context.done();
16  };
17
18  const fetchCheckedCount = async userToken => {
19    const cookie = new tough.Cookie({
20      key: 'user',
21      value: userToken,
22      path: '/',
23      domain: 'techbookfest.org'
```

コード(2)

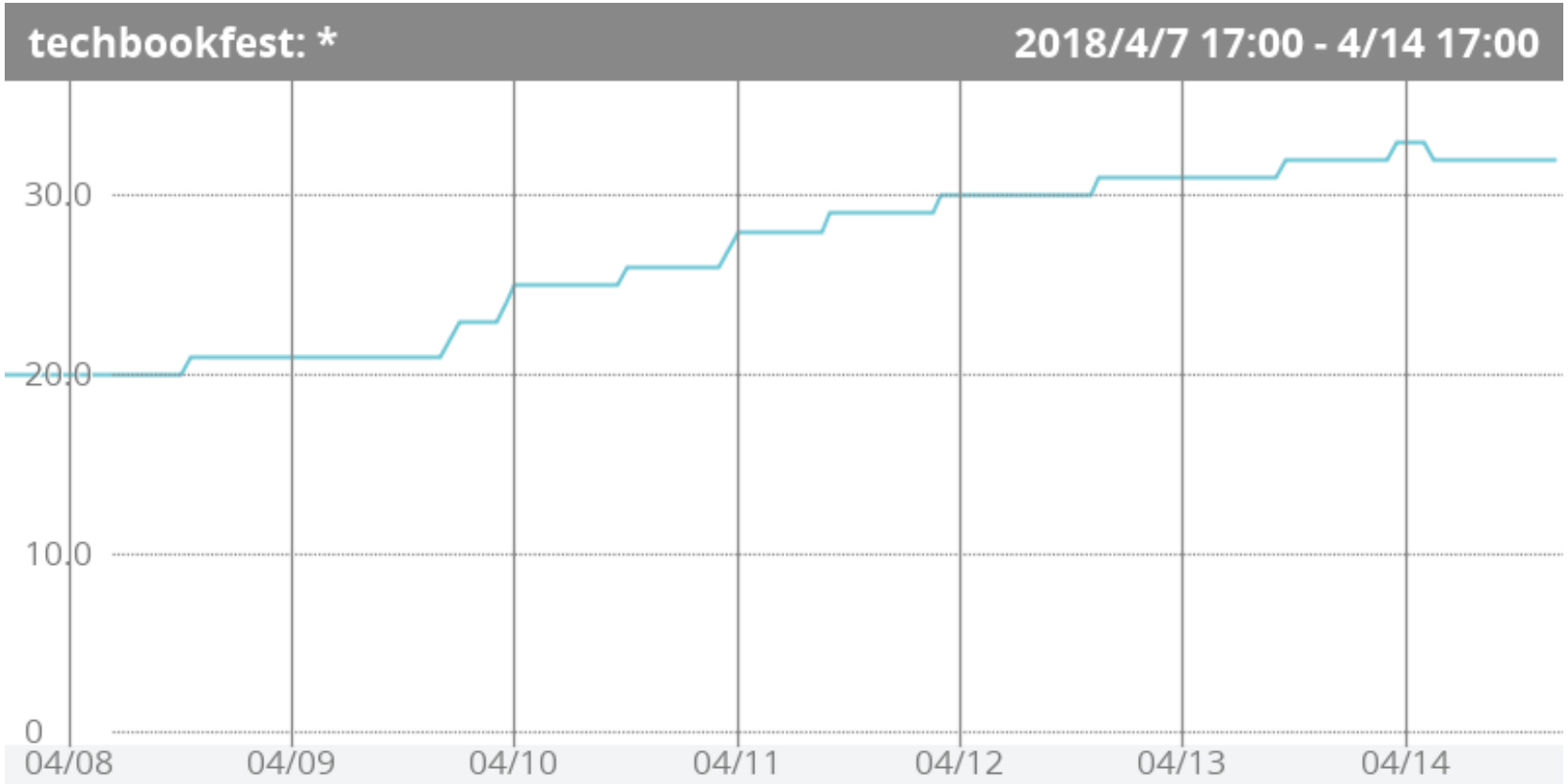
```
const sendToMackerel = async checkedCount => {
    const opt = {
        method: 'POST',
        uri: 'https://api.mackerelio.com/api/v0/services/techbookfest/tsdb',
        body: JSON.stringify([{ name: "circleCheck", time: Math.floor(new Date().getTime() / 1000),
        headers: {
          'content-type': 'application/json',
          'X-Api-Key': apiKey
        },
        resolveWithFullResponse: true
      }
    await rp(opt)
}

const sendToSlack = async message => {
    const opt = {
```

0 0 TSH Resolver 行 58、列 1 スペース: 2 UTF-8 CRLF TypeScript 2

Copyright 2017 Hiroyuki Onaka

作成されるグラフ



Copyright 2017 Hiroyuki Onaka

ありがとうございました！

- 大中浩行(Onaka,Hiroyuki)
- @setoazusa
- グロースエクスパートナーズ株式会社
  アーキテクチャソリューション部
  テクニカルリード
- http://hiroyuki.fieldnotes.jp/

Copyright 2017 Hiroyuki Onaka